SET 20

PART A — Short questions and answers

Q1. Interpret an ER diagram in which MEMBER and TRAINER are connected by an M:N relationship.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Compare the CHAR and VARCHAR2 data types for storing the Phone attribute.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Explain a trivial functional dependency with an example from the MEMBER relation.

Answer: Book_ID → Title means each Book_ID determines exactly one book title.

Q4. Explain how NOT IN with a subquery finds members who have no subscription.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Describe the purpose of the AVG function applied to MonthlyFee.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Explain the benefit of using a stored procedure instead of repeating SQL statements.

Answer: A stored procedure is a named PL/SQL program saved in the database. It can accept parameters and perform reusable operations.

Q7. Compare the terms table, row and column with collection, document and field.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q8. Explain the purpose of the find() method without any argument.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Explain what the following MongoDB query returns: db.members.find({Age: {$gte: 18, $lte: 30}})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Describe the output of: MATCH (t:Trainer)-[:CONDUCTS]->(c:Class) RETURN t.name, count(c);

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A fitness centre maintains a Gym Membership Management System to store member details,

1(a) ER design:

• **MEMBER**(MemberID, MemberName, Age, Phone); primary key: **MemberID**.

• **MEMBERSHIP_PLAN**(PlanID, PlanName, MonthlyFee, DurationMonths); primary key: **PlanID**.

• **SUBSCRIPTION**(SubscriptionID, MemberID, PlanID, StartDate, EndDate); primary key: **SubscriptionID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE MEMBER (
  MemberID NUMBER PRIMARY KEY,
  MemberName VARCHAR2(100),
  Age NUMBER,
  Phone VARCHAR2(100)
);
```

```
CREATE TABLE MEMBERSHIP_PLAN (
  PlanID NUMBER PRIMARY KEY,
  PlanName VARCHAR2(100),
  MonthlyFee NUMBER,
  DurationMonths NUMBER
);
CREATE TABLE SUBSCRIPTION (
  SubscriptionID NUMBER PRIMARY KEY,
  MemberID NUMBER,
  PlanID NUMBER,
  StartDate DATE,
  EndDate DATE
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM SUBSCRIPTION x
JOIN MEMBER a ON x.MemberID = a.MemberID
JOIN MEMBERSHIP_PLAN b ON x.PlanID = b.PlanID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```